CLA 4 – ASSIGNMENT REPORT

ML Model Deployment Strategies and Model Serving using FastAPI

Submitted By:
Name: Perni.Divya Sai Charishma
Roll Number: 231FA04D81
Department: CSE
Course: B.Tech

1. INTRODUCTION:

Machine Learning (ML) models are used to analyze data and make predictions. However, training a model alone is not sufficient. The trained model must be deployed so that users and applications can access its predictions.

This assignment demonstrates the process of training, saving, and deploying a machine learning model using different inference strategies. The Titanic dataset from Kaggle is used to develop a classification model that predicts passenger survival.

The trained model is saved and reused for online, batch, and streaming inference. FastAPI is used to expose the model through REST API endpoints.

2. OBJECTIVES:

The main objectives of this assignment are:

- To train and evaluate a machine learning classification model using the Titanic dataset.
- To save the trained model for future use.
- To perform online inference for one unseen passenger record.
- To perform batch inference for 20 unseen passenger records.
- To implement streaming inference for 10 records with a 1-second delay.
- To deploy the trained model using FastAPI.
- To validate incoming data and test API endpoints.

3. DATASET DESCRIPTION:

The Titanic dataset is obtained from Kaggle.

Dataset Name: Titanic – Machine Learning from Disaster
Dataset File: train.csv
Dataset Link: https://www.kaggle.com/competitions/titanic/data
Task Type: Binary Classification
Target Variable: Survived

The dataset contains passenger details such as age, gender, passenger class, and other information. These features are used to train a classification model to predict whether a passenger survived or not.

4. METHODOLOGY:

The following steps are performed in this assignment:

1. Load the Titanic dataset.
2. Perform data preprocessing.
3. Split the dataset into training and testing portions.

4. Train a suitable classification model.
5. Evaluate the model using accuracy.
6. Save the complete trained pipeline as model.pkl.
7. Perform online inference using one unseen record.
8. Perform batch inference using 20 unseen records.
9. Perform streaming inference using 10 records with a 1-second delay.
10. Develop a FastAPI application for model serving.
11. Test the API endpoints using Swagger UI and a Python client.

5. IMPLEMENTATION:

5.1 Model Training

The Titanic dataset is loaded and preprocessed. The dataset is divided into training and testing portions. A classification model is trained using the training data and evaluated using accuracy.

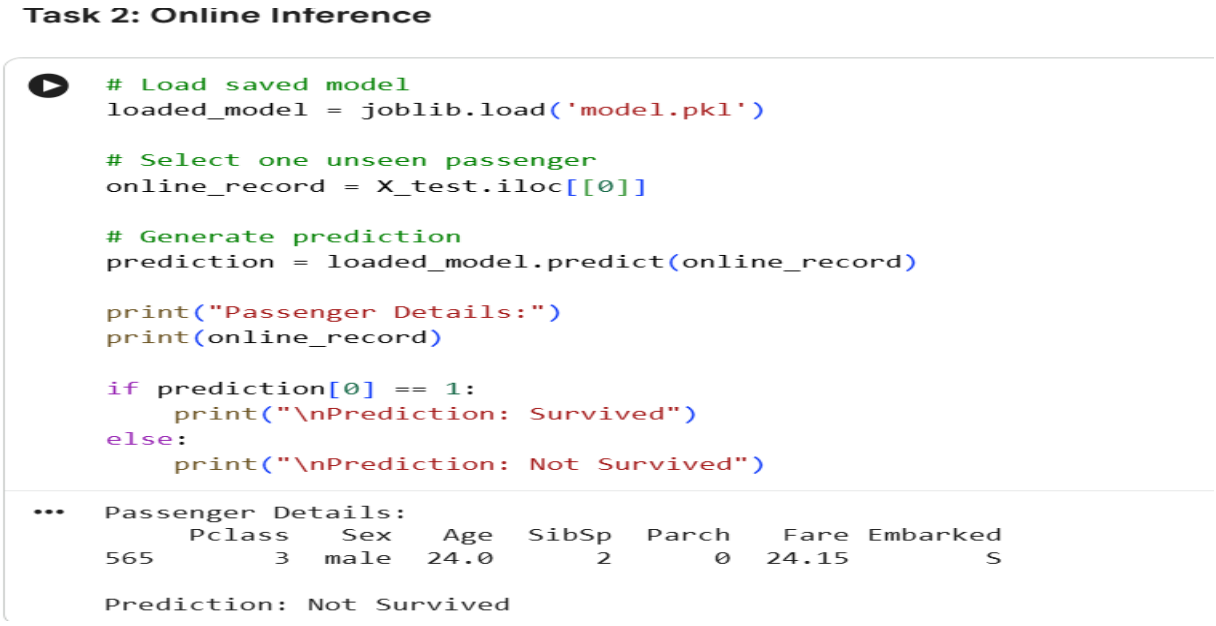
The complete trained pipeline is saved as model.pkl so that it can be reused for all inference strategies.

5.2 Online Inference

Online inference is performed using one unseen passenger record. The saved model is loaded and used to generate a prediction for the input record.

Result: The model successfully generates a prediction for a single unseen record.

Screenshot:



Task 2: Online Inference

```
# Load saved model
loaded_model = joblib.load('model.pkl')

# Select one unseen passenger
online_record = X_test.iloc[[0]]

# Generate prediction
prediction = loaded_model.predict(online_record)

print("Passenger Details:")
print(online_record)

if prediction[0] == 1:
    print("\nPrediction: Survived")
else:
    print("\nPrediction: Not Survived")
```

```
Passenger Details:
     Pclass   Sex   Age  SibSp  Parch   Fare Embarked
565       3  male  24.0      2      0  24.15        S

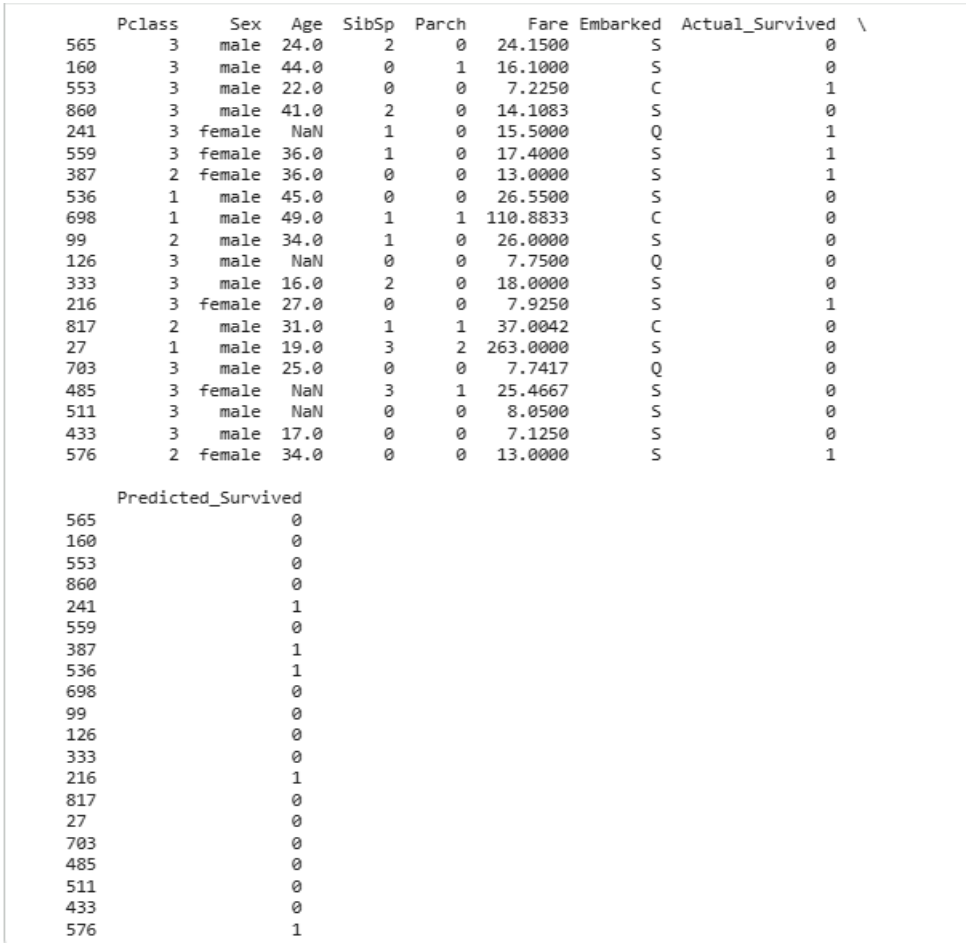
Prediction: Not Survived
```

5.3 Batch Inference

Batch inference is performed using 20 unseen passenger records. All records are processed together, and the generated predictions are saved in batch_predictions.csv.

Result: Predictions are generated for 20 records and saved in a CSV file.

Screenshot:



	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Actual_Survived	Predicted_Survived
565	3	male	24.0	2	0	24.1500	S	0	0
160	3	male	44.0	0	1	16.1000	S	0	0
553	3	male	22.0	0	0	7.2250	C	1	0
860	3	male	41.0	2	0	14.1083	S	0	0
241	3	female	NaN	1	0	15.5000	Q	1	1
559	3	female	36.0	1	0	17.4000	S	1	0
387	2	female	36.0	0	0	13.0000	S	1	1
536	1	male	45.0	0	0	26.5500	S	0	1
698	1	male	49.0	1	1	110.8833	C	0	0
99	2	male	34.0	1	0	26.0000	S	0	0
126	3	male	NaN	0	0	7.7500	Q	0	0
333	3	male	16.0	2	0	18.0000	S	0	0
216	3	female	27.0	0	0	7.9250	S	1	1
817	2	male	31.0	1	1	37.0042	C	0	0
27	1	male	19.0	3	2	263.0000	S	0	0
703	3	male	25.0	0	0	7.7417	Q	0	0
485	3	female	NaN	3	1	25.4667	S	0	0
511	3	male	NaN	0	0	8.0500	S	0	0
433	3	male	17.0	0	0	7.1250	S	0	0
576	2	female	34.0	0	0	13.0000	S	1	1

5.4 Streaming Inference

Streaming inference is simulated using 10 unseen passenger records. The records are processed one at a time with a 1-second delay between each record.

Result: The model generates predictions sequentially for the incoming records.

Screenshot:

```
Streaming Inference Started
...
Incoming Passenger: 1
Prediction: Not Survived
------------------------------
Incoming Passenger: 2
Prediction: Survived
------------------------------
Incoming Passenger: 3
Prediction: Not Survived
------------------------------
Incoming Passenger: 4
Prediction: Survived
------------------------------
Incoming Passenger: 5
Prediction: Not Survived
------------------------------
Incoming Passenger: 6
Prediction: Survived
------------------------------
Incoming Passenger: 7
Prediction: Not Survived
------------------------------
Incoming Passenger: 8
Prediction: Not Survived
------------------------------
Incoming Passenger: 9
Prediction: Not Survived
------------------------------
Incoming Passenger: 10
Prediction: Not Survived
------------------------------
Streaming Inference Completed!
```

5.5 Model Serving Using FastAPI

FastAPI is used to create a REST API for serving the trained machine learning model.

The application contains the following endpoints:

GET /health

This endpoint checks whether the API application is running.

POST /predict

This endpoint accepts passenger details and returns the predicted survival outcome.

Pydantic is used to validate incoming data. The API is tested using Swagger UI and a Python client.

Screenshot: FastAPI Health Endpoint

```
import requests

response = requests.get("http://127.0.0.1:8000/health")

print(response.status_code)
print(response.json())
```

```
200
{'status': 'API is running'}
```

Screenshot: FastAPI Prediction Endpoint

```
import requests

url = "http://127.0.0.1:8000/predict"

passenger_data = {
     "Pclass": 3,
     "Sex": "male",
     "Age": 25,
     "SibSp": 0,
     "Parch": 0,
     "Fare": 7.25,
     "Embarked": "S"
}

response = requests.post(
     url,
     json=passenger_data
)

print("Status Code:", response.status_code)
print("Prediction:", response.json())
```

```
Status Code: 200
Prediction: {'prediction': 'Not Survived', 'survived': 0}
```

Screenshot: Invalid Input Validation

```
invalid_data = {
    "Pclass": 3,
    "Sex": "male",
    "Age": -5,
    "SibSp": 0,
    "Parch": 0,
    "Fare": 7.25,
    "Embarked": "S"
}

response = requests.post(
    url,
    json=invalid_data
)

print("Status Code:", response.status_code)
print("Response:", response.json())
```

```
Status Code: 422
Response: {'detail': [{'type': 'greater_than_equal', 'loc': ['body', 'Age'], 'msg': 'Input should be greater than or equal to 0', 'input': -5, 'ctx': {'ge':
```

6. TASK 6 – OBSERVATIONS:

1. How does Online Inference differ from Batch Inference?

Online inference generates a prediction for one input record when it is requested. Batch inference processes a group of records together and generates predictions for all records in that batch.

2. What is the difference between Streaming Inference and Batch Inference?

Streaming inference processes incoming records one at a time as they arrive, typically with low delay. Batch inference collects multiple records and processes them together at once.

3. Why is FastAPI used for model serving?

FastAPI is used to expose the trained machine learning model through REST API endpoints. It supports request and response handling, input validation using Pydantic, and interactive API testing through Swagger UI.

4. Why should the same trained model be reused for all inference strategies?

Reusing the same saved model keeps predictions consistent across online, batch, streaming, and API inference. It also avoids retraining for each strategy and ensures that the same learned parameters and preprocessing steps are applied.

7. CONCLUSION:

This assignment demonstrates the complete workflow of machine learning model training, saving, and deployment.

The trained model is reused for online, batch, and streaming inference. Batch predictions are saved in a CSV file. FastAPI is used to serve predictions through REST API endpoints, and Pydantic is used for input validation.

The project provides practical understanding of machine learning deployment strategies and model serving using FastAPI.

8. PROJECT FILES:

The following files are included in the project:

- model.pkl – Saved machine learning model.
- app.py – FastAPI application.
- client.py – Python client for API testing.
- batch_predictions.csv – Batch inference results.
- requirements.txt – Required Python packages.
- Google Colab Notebook – Complete implementation and outputs.
- Screenshots – Inference and API testing results.